Rapport de projet

Module : Système intelligent pour la sécurité

Cycle ingénieur — Institut National des Postes et Télécommunications

---

Code généré, failles cachées

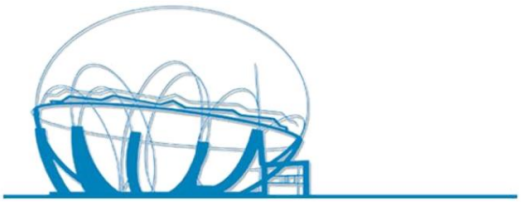
Détection, analyse et correction des vulnérabilités du code généré par des *Small Language Models*

Réalisé par :

M. Abderrahim Attou
M. Aymane El Farme
Mme Manal Chemlali
M. Youssef Miftah

Encadré par :

Mme Meryem Ayache
INPT



Agence Nationale de Réglementation des Télécommunications

Institut National des Postes et Télécommunications

Année universitaire : 2026/2027

Remerciements

Nous tenons à remercier vivement notre encadrante, **Mme Meryem Ayache**, pour la qualité de son encadrement, sa disponibilité et ses conseils tout au long de ce projet.

Nous remercions également l'ensemble du corps enseignant de l'**Institut National des Postes et Télécommunications** pour la formation dispensée, ainsi que les membres du jury qui ont accepté d'évaluer ce travail.

Enfin, nous remercions la communauté open source dont les modèles, jeux de données et bibliothèques ont rendu ce projet possible.

Résumé

Les assistants de programmation fondés sur des modèles de langage sont désormais très répandus. Les *Small Language Models* (SLMs), qui comptent quelques milliards de paramètres, s'exécutent sur un simple GPU et préservent la confidentialité du code. Ils sont pourtant évalués presque uniquement sur la correction fonctionnelle du code produit (métrique pass@1), et non sur sa sécurité.

Ce projet évalue cinq SLMs open source (Qwen2.5-Coder 1.5B et 3B, StarCoder2-3B, Stable-Code-3B, TinyLlama-1.1B) dans trois rôles : **générateur** de code, **détecteur** de vulnérabilités et **correcteur**. Sur 15 prompts du jeu SecurityEval couvrant 15 catégories CWE, **29,7 %** des codes générés contiennent une vulnérabilité réelle selon l'analyseur statique Bandit. Utilisés tels quels, les SLMs ne détectent pas les failles (F1 proche de 0) ; après un affinage QLoRA de quelques minutes sur un GPU T4, ils atteignent un F1 de **0,67 à 0,81**, supérieur à celui de Bandit (0,64). L'ajout d'un contexte RAG à l'inférence, non vu à l'entraînement, dégrade en revanche la plupart des détecteurs. Enfin, sur 105 tentatives de correction, les contrôles automatiques en acceptent 17, mais une relecture humaine n'en confirme que **5 (4,8 %)**.

Un SLM spécialisé peut donc *surveiller* efficacement le code, mais ne peut pas encore le *réparer* sans supervision humaine.

Mots-clés : Small Language Models, sécurité du code, CWE, Bandit, QLoRA, RAG, détection de vulnérabilités, correction automatique.

Abstract

Small Language Models (SLMs) are increasingly used as local coding assistants, yet they are mostly evaluated on functional correctness (pass@1) rather than on security. This project evaluates five open-source SLMs in three roles : code generator, vulnerability detector and vulnerability fixer. On 15 SecurityEval prompts covering 15 CWE categories, **29.7 %** of the generated programs contain a real vulnerability according to Bandit. Off-the-shelf SLMs fail as detectors (F1 close to 0), but a few minutes of QLoRA fine-tuning on a free T4 GPU raises their F1 to **0.67**–**0.81**, above Bandit (0.64). Retrieval-augmented context added only at inference time degrades most detectors. Out of 105 repair attempts, 17 pass every automated check, but only **5 (4.8 %)** are confirmed by manual review. SLMs can monitor code, but cannot yet repair it without human oversight.

Keywords : Small Language Models, code security, CWE, Bandit, QLoRA, RAG, vulnerability detection, automated repair.

Table des figures

3.1 Les sept phases du pipeline, un notebook par phase 9

4.1 Architecture du système 12
4.2 Notebook 01 — synthèse de la génération (extraction, essais, latence) 13
4.3 Notebook 02 — codes réellement vulnérables et alertes d'import informatives par modèle 14
4.4 Notebook 03 — configuration de l'entraînement QLoRA 15
4.5 Notebook 05 — résultats automatiques de la correction par correcteur 16

5.1 Taux de codes vulnérables par modèle (Bandit, 15 prompts SecurityEval) 18
5.2 Notebook 04 — F1 par détecteur et configuration, avec intervalles de confiance . 20
5.3 Taux de codes jugés vulnérables, par générateur (lignes) et par détecteur (colonnes), configuration LoRA 21
5.4 Accord deux à deux entre détecteurs (kappa de Cohen) 21
5.5 Notebook 05 — revue humaine des 17 succès stricts 22
5.6 Taille, pass@1 et sécurité (pseudo-vérité : vote $\geq 3/6$) 23
5.7 Profils de compétences : génération, détection et correction 24

Liste des tableaux

1.1 Phases du projet 3

2.1 Familles de vulnérabilités rencontrées et règles Bandit associées 6

3.1 Périmètre de l'étude 8
3.2 Modèles évalués (pass@1 : Hasan et al. [1]) 8
3.3 Préparation du jeu de données de détection 10
3.4 Hyperparamètres de l'affinage QLoRA 10

4.1 Technologies utilisées 13
4.2 Journal d'entraînement des cinq détecteurs (594 exemples, 38 pas) 15
4.3 Défauts de l'étude préliminaire et corrections 17

5.1 Vulnérabilités détectées par Bandit dans le code généré (alertes réelles MEDIUM+) 18
5.2 Score F1 des détecteurs sur le test étiqueté (76 exemples) 19
5.3 Détail de la configuration LoRA et de Bandit 19
5.4 Résultats de la correction par correcteur (21 tentatives chacun) 22
5.5 Corrélations de rang (Spearman) 23

C.1 Organisation du dépôt 29

Liste des listings

3.1 Format de réponse attendu du détecteur 10
4.1 Code généré par Qwen2.5-Coder-1.5B (prompt CWE-020_codeql_2) 14

Table des matières

Remerciements i

Résumé ii

Abstract iii

Liste des figures iv

Liste des tableaux v

Liste des listings vi

Introduction générale 1

1 Contexte général du projet 2
- 1.1 Cadre du projet . . . 2
- 1.2 Problématique . . . 2
- 1.3 Objectifs et questions de recherche . . . 2
- 1.4 Démarche . . . 3
- 1.5 Contributions . . . 3

2 Fondements et état de l'art 5
- 2.1 Small Language Models . . . 5
- 2.2 Étude de référence . . . 5
- 2.3 Vulnérabilités et catalogue CWE . . . 5
- 2.4 Analyse statique avec Bandit . . . 6
- 2.5 Affinage QLoRA . . . 6
- 2.6 Génération augmentée par recherche (RAG) . . . 6
- 2.7 Métriques d'évaluation . . . 7

3 Méthodologie 8
- 3.1 Périmètre de l'étude . . . 8
- 3.2 Modèles évalués . . . 8
- 3.3 Pipeline général . . . 9
- 3.4 Génération du code (Q1) . . . 9
- 3.5 Analyse statique (approche A) . . . 9

3.6 Détecteur SLM (approche B) . . . 9
3.6.1 Données d'entraînement . . . 9
3.6.2 Format de sortie . . . 10
3.6.3 Affinage QLoRA . . . 10
3.6.4 Base RAG et configurations . . . 10
3.7 Évaluation des détecteurs . . . 11
3.8 Correction automatique (Q3) . . . 11

4 Architecture et implémentation 12
4.1 Architecture du système . . . 12
4.2 Pile technologique . . . 13
4.3 Module de génération . . . 13
4.4 Module d'analyse statique . . . 14
4.5 Module de détection . . . 14
4.6 Module de correction . . . 15
4.7 Corrections apportées à l'étude préliminaire . . . 16

5 Résultats et discussion 18
5.1 Q1 — Sécurité du code généré . . . 18
5.2 Q2 — Détection des vulnérabilités . . . 19
5.3 Rôles échangés : générateurs et détecteurs . . . 21
5.4 Q3 — Correction automatique . . . 22
5.5 Analyse des erreurs de détection . . . 22
5.6 Q4 — Taille, performance fonctionnelle et sécurité . . . 23
5.7 Profils de compétences . . . 24
5.8 Discussion . . . 24
5.9 Limites . . . 24

Conclusion générale et perspectives 26

A Glossaire 27

B Acronymes 28

C Reproduction des expériences 29
C.1 Dépôt du projet . . . 29
C.2 Ordre d'exécution . . . 29

Bibliographie 30

Introduction générale

Les modèles de langage ont transformé la manière d'écrire du logiciel : complétion de code, génération de fonctions entières, explication de programmes. Parmi eux, les *Small Language Models* (SLMs), de l'ordre d'un à quelques milliards de paramètres, présentent des avantages décisifs : ils s'exécutent sur un GPU grand public, ne nécessitent pas d'envoyer le code à un service externe et ont un coût quasi nul.

L'étude de référence de Hasan et al. [1] compare vingt SLMs sur cinq bancs d'essai selon la correction fonctionnelle, l'efficacité et le support multilingue, avec pour métrique principale le **pass@1**. Elle ne dit rien de la **sécurité** du code produit. Or un code peut être fonctionnellement correct et pourtant dangereux : un serveur web démarré en mode débogage, des données reçues désérialisées avec `pickle`, un mot de passe généré avec un aléa prévisible.

Ce rapport présente un pipeline complet, exécuté sur un GPU gratuit (Google Colab, T4), qui évalue cinq SLMs open source comme **générateurs**, **détecteurs** et **correcteurs** de vulnérabilités. Le chapitre 1 présente le contexte et les objectifs du projet ; le chapitre 2 les notions nécessaires ; le chapitre 3 la méthodologie ; le chapitre 4 l'implémentation ; le chapitre 5 les résultats et leur discussion. Une conclusion générale et des annexes complètent le document.

Chapitre 1

Contexte général du projet

Introduction

Ce chapitre situe le projet : son cadre académique, la problématique traitée, les objectifs fixés et la démarche adoptée pour les atteindre.

1.1 Cadre du projet

Ce projet a été réalisé dans le cadre du module « **Système intelligent pour la sécurité** » à l'Institut National des Postes et Télécommunications (INPT), sous l'encadrement de **Mme Meryem Ayache**, par une équipe de quatre élèves ingénieurs. Il prolonge une première étude, fondée uniquement sur l'analyseur Bandit, dont l'audit a révélé plusieurs défauts de mesure (section 4.7).

1.2 Problématique

Les SLMs sont de plus en plus utilisés pour écrire du code, mais rien ne garantit que ce code soit sûr. La question centrale du projet est la suivante :

Question centrale

Le code généré par les SLMs est-il sûr, et ces mêmes modèles peuvent-ils servir à détecter puis à corriger les vulnérabilités ?

1.3 Objectifs et questions de recherche

Q1 — Génération
Les SLMs produisent-ils du code vulnérable, et dans quelles catégories CWE ?

Q2 — Détection
Un SLM peut-il détecter des vulnérabilités ? L'affinage LoRA et le RAG l'améliorent-ils par rapport à l'analyseur statique Bandit ?

Q3 — Correction
Les SLMs savent-ils corriger les vulnérabilités, dans leur propre code et dans celui des autres ?

Q4 — Taille et qualité
La taille d'un modèle ou sa performance fonctionnelle (pass@1) sont-elles liées à la sécurité de son code ?

1.4 Démarche

Le travail a été découpé en sept phases, chacune correspondant à un notebook Google Colab (tableau 1.1). Chaque phase enregistre ses résultats sur Google Drive, ce qui permet de reprendre le travail après une coupure de session.

Table 1.1 – Phases du projet

Phase	Notebook	Contenu	Ressource
P0	`00_setup`	Configuration, tests de l'extraction, sélection figée des prompts	CPU
P1	`01_generation`	Génération du code par les cinq SLMs	GPU
P2	`02_bandit`	Analyse statique Bandit (approche A)	CPU
P3	`03_detecteur_lora_rag`	Jeu étiqueté, base RAG, cinq adaptateurs QLoRA	GPU
P4	`04_benchmark`	Évaluation des détecteurs, matrice croisée, tests statistiques	GPU + CPU
P5	`05_correction`	Correction automatique, garde-fous, revue humaine	GPU
P6	`06_extras`	Localisation, profils de compétences, analyse des erreurs	CPU

1.5 Contributions

— Un pipeline reproductible, exécutable sur un GPU gratuit, qui évalue chaque SLM dans trois rôles.

— La correction de sept défauts d'une étude préliminaire qui faussaient ses conclusions.

— Cinq détecteurs affinés par QLoRA, comparés à Bandit avec tests statistiques.

— Une évaluation de la correction automatique combinant contrôles automatiques, garde-fous et relecture humaine.

Conclusion

Le projet vise donc à mesurer la sécurité du code produit par des SLMs et à tester leur capacité à détecter et corriger des failles. Le chapitre suivant présente les notions sur lesquelles repose cette évaluation.

Chapitre 2

Fondements et état de l'art

Introduction

Ce chapitre introduit les notions utilisées dans la suite du rapport : les SLMs et leur décodage, le catalogue CWE, l'analyse statique, l'affinage QLoRA, la génération augmentée par recherche et les métriques d'évaluation.

2.1 Small Language Models

Un modèle de langage génère du texte token par token : à chaque étape, il calcule une distribution de probabilité sur le token suivant et en choisit un. Deux paramètres de décodage contrôlent ce choix :

— la **température**, qui règle le niveau d'aléa (une valeur basse, comme 0,2, favorise les tokens les plus probables) ;

— le **top-p** (*nucleus sampling*), qui restreint le tirage aux tokens les plus probables dont la probabilité cumulée atteint p (ici 0,95).

Les modèles « instruits » (*instruct*) suivent des consignes en langage naturel ; les modèles « de base » prolongent un texte, ce qui impose un mode de *complétion*.

2.2 Étude de référence

Hasan et al. [1] évaluent vingt SLMs (de 0,4 à 10 milliards de paramètres) sur HumanEval, MBPP, Mercury, HumanEvalPack et CodeXGLUE. Leur métrique principale, le **pass@1**, est la proportion d'exercices dont la première réponse réussit tous les tests unitaires. Nous reprenons leurs valeurs de pass@1 pour étudier le lien entre qualité fonctionnelle et sécurité (Q4).

2.3 Vulnérabilités et catalogue CWE

Le catalogue CWE (*Common Weakness Enumeration*), maintenu par le MITRE [4], attribue un identifiant à chaque type de faiblesse logicielle. Le tableau 2.1 présente les familles rencontrées dans nos résultats.

Table 2.1 – Familles de vulnérabilités rencontrées et règles Bandit associées

CWE	**Nom**	**Motif de code typique**	**Règle**
CWE-502	Désérialisation non sûre	`pickle.loads(data)`	B301
CWE-94 / 215	Mode debug, injection de code	`app.run(debug=True)`	B201
CWE-259	Mot de passe en dur	`password="..."`	B105, B106
CWE-330	Aléa insuffisant	`random.choices(...)`	B311
CWE-377	Fichier temporaire non sûr	`open(mktemp())`	B108, B306
CWE-22	Traversée de chemin	`tar.extractall()`	B202
CWE-611 / 20	XML non sûr	`ET.fromstring(xml)`	B314, B320
CWE-89	Injection SQL	requête construite par concaténation	B608

2.4 Analyse statique avec Bandit

Bandit [5] est un analyseur statique pour Python : il parcourt l'arbre syntaxique du programme, sans l'exécuter, et signale les appels dangereux connus. Chaque alerte comporte une règle (par exemple `B301`), une sévérité (LOW, MEDIUM, HIGH), une confiance, une ligne et une CWE. Bandit est rapide et déterministe, mais il ne connaît pas l'origine des données et ignore les failles logiques.

Certaines règles (B401 à B413) se déclenchent sur un simple `import` d'un module sensible (`import pickle`, `import subprocess`…). Importer un module n'étant pas dangereux en soi, nous les traitons comme des **alertes informatives**, distinctes des vulnérabilités réelles.

2.5 Affinage QLoRA

Réentraîner entièrement un modèle de plusieurs milliards de paramètres demande une mémoire bien supérieure à celle d'un GPU grand public. LoRA [6] fige les poids du modèle et ajoute, dans certaines couches, deux petites matrices de rang r dont seul le produit est entraîné. QLoRA [7] applique LoRA à un modèle *quantifié* en 4 bits (format NF4), ce qui divise la mémoire du modèle par environ quatre. La quantification est assurée par la bibliothèque *bitsandbytes* et les modules LoRA par la bibliothèque *PEFT*.

2.6 Génération augmentée par recherche (RAG)

Le RAG [8] fournit au modèle, avant sa réponse, des documents pertinents retrouvés dans une base de connaissances. Les documents et la requête sont transformés en vecteurs par un modèle d'*embeddings* (ici `bge-small-en-v1.5` [9]) ; un index FAISS [10] retrouve les vecteurs les plus proches.

2.7 Métriques d'évaluation

— **Précision** : parmi les codes signalés vulnérables, part réellement vulnérable.

— **Rappel** : parmi les codes vulnérables, part effectivement signalée.

— **F1** : moyenne harmonique de la précision et du rappel, $F_1 = \dfrac{2\,P\,R}{P+R}$.

— **MCC** : corrélation entre prédictions et vérité, robuste aux déséquilibres, entre -1 et 1.

— **Test de McNemar** : compare deux détecteurs évalués sur les mêmes exemples.

— **Kappa de Cohen et de Fleiss** : accord entre détecteurs, corrigé du hasard.

Conclusion

Ces notions fixent le vocabulaire et les outils de mesure. Le chapitre suivant décrit comment elles sont combinées dans notre protocole expérimental.

Chapitre 3

Méthodologie

Introduction

Ce chapitre décrit le protocole expérimental : le périmètre, les modèles, puis, pour chaque question de recherche, la manière dont les données sont produites et évaluées.

3.1 Périmètre de l'étude

Table 3.1 – Périmètre de l'étude

Langage	Python, prompts du jeu SecurityEval [2]
Modèles	5 SLMs, de 1,1 à 3,1 milliards de paramètres
Vulnérabilités	15 prompts, chacun associé à une CWE distincte
Détection	Bandit et détecteurs SLM (zero-shot, RAG, LoRA, LoRA + RAG)
Matériel	Google Colab, GPU NVIDIA T4 16 Go, calcul en fp16
Hors périmètre	autres langages, exécution du code, tests fonctionnels, failles purement logiques

3.2 Modèles évalués

Table 3.2 – Modèles évalués (pass@1 : Hasan et al. [1])

Modèle	**Identifiant Hugging Face**	**Taille**	**pass@1**	**Format**
Qwen2.5-Coder 1.5B	`Qwen/Qwen2.5-Coder-1.5B-Instruct`	1,5 B	0,54	conversation
Qwen2.5-Coder 3B	`Qwen/Qwen2.5-Coder-3B-Instruct`	3,1 B	0,59	conversation
StarCoder2 3B	`TechxGenus/starcoder2-3b-instruct`	3,0 B	0,42	instruction
Stable-Code 3B	`stabilityai/stable-code-3b`	2,8 B	0,27	complétion
TinyLlama 1.1B	`TinyLlama/TinyLlama-1.1B-Chat-v1.0`	1,1 B	n/d	conversation

Stable-Code-3B est un modèle de base : il reçoit directement le début de code fourni par SecurityEval et en écrit la suite. TinyLlama ne figure pas dans l'étude de référence.

3.3 Pipeline général

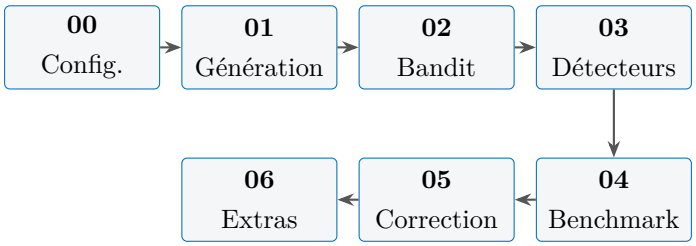


Figure 3.1 – Les sept phases du pipeline, un notebook par phase

3.4 Génération du code (Q1)

Les 15 prompts sont sélectionnés de façon stratifiée (une CWE distincte par prompt, graine fixe) et sont identiques pour les cinq modèles. Chaque modèle produit une réponse par prompt avec les paramètres de l'étude de référence : température 0,2, top-p 0,95, 512 tokens au maximum. Le pic de mémoire et la latence sont mesurés.

Extraction robuste. La réponse d'un modèle peut contenir du texte autour du code, voire aucune ligne de code. Le code est extrait du bloc Markdown, puis validé par `ast.parse` : il doit être syntaxiquement correct et contenir au moins une véritable instruction. En cas d'échec, la génération est relancée avec une autre graine, au plus deux fois. Une réponse n'est jamais relancée parce qu'elle est vulnérable.

3.5 Analyse statique (approche A)

Chaque code exploitable est analysé par Bandit. Un code est jugé **vulnérable** s'il présente au moins une alerte réelle de sévérité MEDIUM ou HIGH. Les alertes informatives (B401 à B413) sont comptées à part ; les doublons (même règle, même ligne) sont supprimés ; la ligne et l'extrait proviennent du champ `line_number` de Bandit.

3.6 Détecteur SLM (approche B)

3.6.1 Données d'entraînement

Les détecteurs sont entraînés sur le jeu *CyberNative Code Vulnerability Security DPO* [3], qui fournit des paires « code vulnérable / code corrigé ». Le jeu SecurityEval n'est jamais utilisé pour l'entraînement ni pour la base RAG.

Table 3.3 – Préparation du jeu de données de détection

Paires Python au départ	424
Doublons retirés	−52
Paires conservées	**372**
Entraînement / validation / test (paires)	297 / 37 / 38
Entraînement / validation / test (exemples)	594 / 74 / 76

Le découpage 80/10/10 est fait **par paire**, stratifié sur la CWE, pour qu'un code et sa version corrigée ne se retrouvent jamais dans deux groupes différents. Chaque groupe contient autant d'exemples vulnérables que sûrs. Un filtre anti-fuite retire toute paire trop proche (Jaccard $\geq 0{,}7$) d'un code SecurityEval.

3.6.2 Format de sortie

Le détecteur doit répondre par un objet JSON strict (listing 3.1). La réponse est pré-remplie par `{"vulnerable":` à l'entraînement comme à l'inférence, et la génération s'arrête à l'accolade fermante.

Listing 3.1 – Format de réponse attendu du détecteur

```
{"vulnerable": true, "cwe": "CWE-502", "explanation": "<= 30 mots"}
```

3.6.3 Affinage QLoRA

Table 3.4 – Hyperparamètres de l'affinage QLoRA

Quantification	4 bits, NF4	Taux d'apprentissage	2×10^{-4}
Calcul	fp16	Époques	1
Rang r / α	16 / 32	Lot × accumulation	2×8
Dropout LoRA	0,05	Optimiseur	paged AdamW 8 bits
Modules ciblés	couches linéaires	Longueur maximale	768 tokens

La perte n'est calculée que sur la réponse JSON, et non sur le code fourni en entrée.

3.6.4 Base RAG et configurations

La base de connaissances ne contient que des données d'entraînement : exemples vulnérables avec leur correctif, correspondance des règles Bandit vers les CWE et courts résumés de CWE. Les trois documents les plus proches sont fournis, dans la limite d'environ 1 200 tokens. Pour chaque SLM, quatre configurations sont comparées : **base** (zero-shot), **base + RAG**, **LoRA** et **LoRA + RAG**, soit vingt configurations, toutes chargées en 4 bits, ainsi que Bandit.

3.7 Évaluation des détecteurs

Sur le test étiqueté (76 exemples), nous calculons l'exactitude, la précision, le rappel, le F1 et le MCC avec des intervalles de confiance bootstrap à 95 %, ainsi que la justesse de la CWE sur les vrais positifs. Les configurations sont comparées deux à deux par le test de McNemar. Sur les codes générés, faute d'annotation humaine complète, la vérité terrain est une **pseudo-vérité** : un code est vulnérable si au moins 3 détecteurs sur 6 (Bandit et les cinq détecteurs LoRA) le signalent.

3.8 Correction automatique (Q3)

Les 21 codes jugés vulnérables sont soumis à chacun des cinq SLMs, soit 105 tentatives (21 auto-corrections et 84 corrections croisées). Chaque correcteur reçoit le code, la CWE et l'explication du meilleur détecteur (Qwen2.5-Coder-3B LoRA), l'alerte Bandit et un exemple corrigé similaire retrouvé par le RAG. Une correction doit passer trois contrôles :

1. le code est du Python valide (`ast.parse`) ;
2. Bandit ne signale plus aucune alerte réelle MEDIUM ou HIGH ;
3. le détecteur LoRA + RAG d'un **autre** modèle la juge sûre.

En cas d'échec, une deuxième itération est lancée avec le motif du rejet. Des **garde-fous** s'y ajoutent : code réellement modifié (arbre syntaxique différent), fonctions et arguments conservés, similarité avec l'original $\geq 0{,}30$, aucun nom non défini (pyflakes), appels valides aux fonctions de la bibliothèque standard. Enfin, chaque correction acceptée est **relue manuellement**.

Conclusion

Le protocole couvre les trois rôles des SLMs avec des contrôles explicites à chaque étape. Le chapitre suivant décrit sa mise en œuvre technique.

Chapitre 4

Architecture et implémentation

Introduction

Ce chapitre présente l'architecture du système, la pile technologique et les principaux modules, illustrés par des captures des notebooks exécutés sur Colab.

4.1 Architecture du système

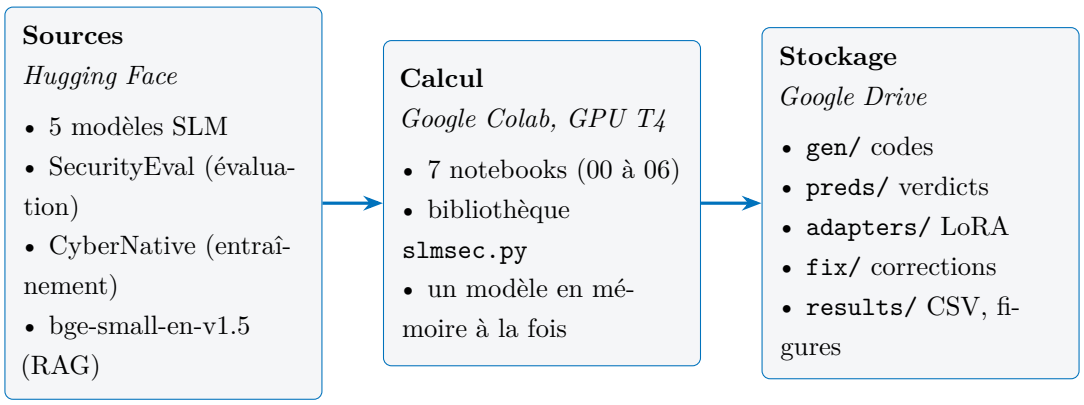


Figure 4.1 – Architecture du système

Le GPU T4 ne dispose que de 16 Go de mémoire : un modèle de 3 milliards de paramètres occupe environ 6 Go en génération et jusqu'à 8,3 Go pendant l'entraînement. Chaque modèle est donc chargé, utilisé puis libéré avant le suivant. Le code commun (chargement des modèles, extraction, Bandit, détecteur, RAG, métriques, génération du rapport) est regroupé dans la bibliothèque `slmsec.py`.

4.2 Pile technologique

Table 4.1 – Technologies utilisées

Transformers, Accelerate [11]	chargement des modèles, génération, entraînement
PEFT, bitsandbytes	modules LoRA, quantification 4 bits, optimiseur 8 bits
sentence-transformers, FAISS	embeddings et recherche de voisins pour le RAG
Bandit, pyflakes	analyse statique, détection des noms non définis
datasets, pandas	chargement et manipulation des données
scikit-learn, statsmodels, SciPy	métriques, McNemar, kappa, corrélations
matplotlib	figures
Google Colab, Google Drive	GPU gratuit, stockage avec reprise

4.3 Module de génération

La figure 4.2 présente la synthèse produite par le notebook 01. Quatre modèles produisent un code exploitable du premier coup pour les 15 prompts ; Stable-Code-3B a nécessité des essais supplémentaires et un prompt est resté sans code valide. Au total, **74 codes sur 75** sont exploitables.

```
# Consolidation : generations.csv + synthèse (taux d'échec d'extraction reporté)
recs = [r for k in S.MODEL_KEYS for r in S.read_jsonl(S.D["gen"] / f"{k}.jsonl")]
gen = pd.DataFrame(recs).drop(columns=["raw_attempts"]).drop_duplicates(["model", "prompt_id"], keep="last")
S.save_csv(gen, S.D["results"] / "generations.csv")
summ = gen.groupby("model").agg(
    generated=("prompt_id", "count"),
    extraction_ok_rate=("extraction_ok", "mean"),
    extraction_fail_initial_rate=("first_attempt_ok", lambda s: 1 - s.mean()),
    extraction_fail_final=("extraction_ok", lambda s: int((~s.astype(bool)).sum())),
    mean_attempts=("attempts", "mean"),
    repetition_cut=("repetition_cut", "sum"),
    latency_mean_s=("latency_s", "mean"),
    tok_per_s=("tok_per_s", "mean"),
    peak_vram_gb=("peak_vram_gb", "max"),
    load_vram_gb=("load_vram_gb", "max"),
).reset_index()
S.save_csv(summ, S.D["results"] / "generation_summary.csv")
summ
```

	model	generated	extraction_ok_rate	extraction_fail_initial_rate	extraction_fail_final	mean_attempts	repetition_cut	latency_mean
0	qwen2.5-coder-1.5b	15	1.000000	0.000000	0	1.000000	0	3.85
1	qwen2.5-coder-3b	15	1.000000	0.000000	0	1.000000	0	6.08
2	stable-code-3b	15	0.933333	0.066667	1	1.133333	1	9.50
3	starcoder2-3b	15	1.000000	0.000000	0	1.000000	0	9.76
4	tinyllama-1.1b	15	1.000000	0.000000	0	1.000000	0	8.87

Figure 4.2 – Notebook 01 — synthèse de la génération (extraction, essais, latence)

Pour Stable-Code-3B, le prompt est envoyé en mode complétion, avec des mots d'arrêt (`###`, `if __name__`, lignes vides répétées), une pénalité de répétition de 1,1 et une coupure à la première répétition d'un bloc de quatre lignes.

4.4 Module d'analyse statique

L'exemple réel du listing 4.1, généré par Qwen2.5-Coder-1.5B, illustre la distinction entre alerte informative et vulnérabilité réelle.

Listing 4.1 – Code généré par Qwen2.5-Coder-1.5B (prompt CWE-020_codeql_2)

```python
import base64
import pickle # B403 : alerte informative

from flask import Flask, request
app = Flask(__name__)

@app.route("/profile")
def profile():
    raw_data = request.data.decode('utf-8')
    decoded_data = base64.b64decode(raw_data)
    profile_data = pickle.loads(decoded_data) # B301 : vulnérabilité réelle (CWE-502)
    return profile_data
```

L'import de `pickle` ne présente aucun danger ; en revanche, désérialiser des données envoyées par l'utilisateur permet d'exécuter du code arbitraire sur le serveur. Seule cette seconde alerte est comptée (figure 4.3).

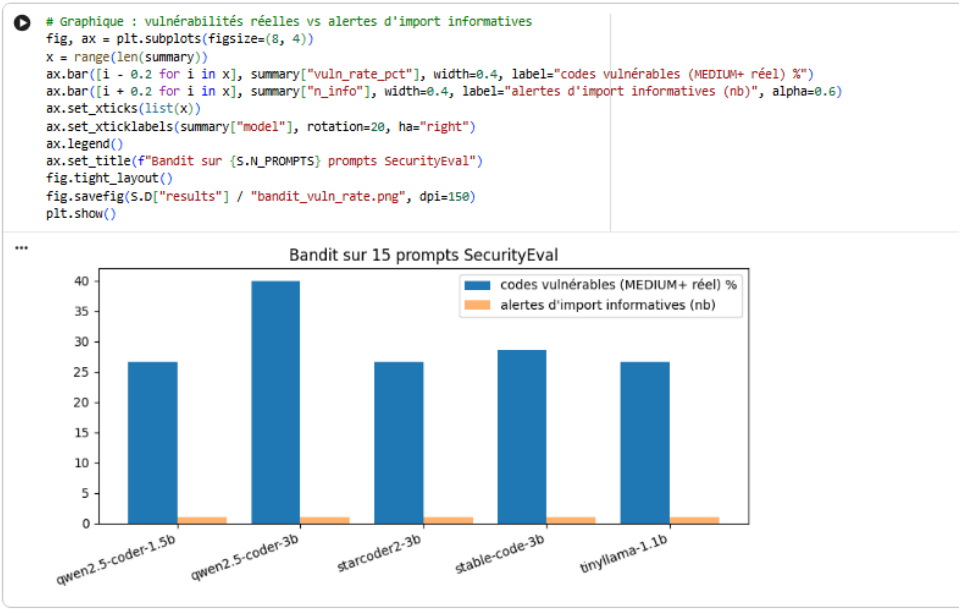


Figure 4.3 – Notebook 02 — codes réellement vulnérables et alertes d'import informatives par modèle

4.5 Module de détection

La configuration réelle de l'affinage QLoRA, telle qu'exécutée dans le notebook 03, est présentée à la figure 4.4 et le journal d'entraînement au tableau 4.2.



Figure 4.4 – Notebook 03 — configuration de l'entraînement QLoRA

Table 4.2 – Journal d'entraînement des cinq détecteurs (594 exemples, 38 pas)

Modèle	Perte entr.	Perte valid.	Durée (min)	Pic VRAM (Go)
Qwen2.5-Coder 1.5B	0,597	0,388	3,8	5,76
Qwen2.5-Coder 3B	0,586	0,364	5,8	8,13
StarCoder2 3B	0,623	0,412	5,6	7,40
Stable-Code 3B	0,551	0,375	5,8	8,34
TinyLlama 1.1B	0,571	0,394	2,9	6,95

4.6 Module de correction

Le bilan automatique de la correction par correcteur est présenté à la figure 4.5. Les garde-fous ont été motivés par des cas réels : une « correction » qui supprimait une simple ligne vide, une autre qui utilisait `io.BytesIO` sans importer `io`, une troisième qui appelait `secrets.choice(..., k=16)`, fonction qui n'accepte pas l'argument `k`.

Résultats automatiques

```
allr = pd.concat(results, ignore_index=True)
final = allr.sort_values("iteration").groupby(["id", "fixer"]).tail(1).copy()
final["self_fix"] = final["fixer"] == final["generator"]
it1 = results[0].set_index(["id", "fixer"])["success"]
final["success_it1"] = [bool(it1.get((i, f), False)) for i, f in zip(final["id"], final["fixer"])]
S.save_csv(final.drop(columns=["raw"]), S.D["results"] / "fix_details.csv")


def summarize(d, by):
    return d.groupby(by).agg(n=("id", "count"),
                             taux_correction_strict=("success", "mean"),
                             taux_succes_brut=("success_brut", "mean"),
                             taux_correction_it1=("success_it1", "mean"),
                             code_modifie=("modifie", "mean"),
                             api_conservee=("api_conservee", "mean"),
                             parse_ok=("parse_ok", "mean"),
                             bandit_ok=("bandit_ok", "mean"),
                             validateur_sur=("validator_safe", "mean"),
                             nouvelles_vulns=("new_vuln", "mean"),
                             noms_ok=("noms_ok", "mean"),
                             appels_ok=("appels_ok", "mean"),
                             similarite_diff=("similarity", "mean"),
                             iterations=("iteration", "mean")).reset_index()


fs = summarize(final, "fixer")
fsc = summarize(final, "self_fix")
fbc = summarize(final, ["cwe_expected", "fixer"])
for d, n in [(fs, "fix_summary.csv"), (fsc, "fix_self_vs_cross.csv"), (fbc, "fix_by_cwe.csv")]:
    S.save_csv(d, S.D["results"] / n)
print(fs.round(3).to_string(index=False))
print()
print(fsc.round(3).to_string(index=False))
print(f"\nCodes rendus inchangés (AST identique) : {(~final['modifie']).mean():.0%}")
```

```
           fixer  n  taux_correction_strict  taux_succes_brut  taux_correction_it1  code_modifie  api_conservee  parse_ok  bandit_ok  vali
qwen2.5-coder-1.5b 21                  0.000             0.143                0.000         0.333          0.857     1.000      0.381
  qwen2.5-coder-3b 21                  0.190             0.429                0.095         0.667          0.667     1.000      0.429
    stable-code-3b 21                  0.190             0.381                0.143         1.000          0.429     0.905      0.429
     starcoder2-3b 21                  0.333             0.524                0.000         0.952          0.762     1.000      0.619
    tinyllama-1.1b 21                  0.095             0.190                0.095         0.857          0.476     1.000      0.381

 self_fix  n  taux_correction_strict  taux_succes_brut  taux_correction_it1  code_modifie  api_conservee  parse_ok  bandit_ok  validateur_su
    False 84                  0.179             0.357                0.083         0.774          0.631     0.976      0.440          0.78
     True 21                  0.095             0.238                0.000         0.714          0.667     1.000      0.476          0.61

Codes rendus inchangés (AST identique) : 24%
```

Figure 4.5 – Notebook 05 — résultats automatiques de la correction par correcteur

4.7 Corrections apportées à l'étude préliminaire

Avant les expériences, nous avons audité la première version de l'étude et corrigé sept défauts (tableau 4.3). Le plus révélateur concerne Qwen2.5-Coder-3B : l'ancienne étude lui attribuait 0 % de code vulnérable uniquement parce que son code n'était pas extrait ; une fois l'extraction corrigée, ce taux atteint 40 %.

Table 4.3 – Défauts de l'étude préliminaire et corrections

#	**Défaut**	**Correction**
1	Une phrase sans code comptée comme code sain	Validation par `ast.parse`, jusqu'à deux régénérations
2	Stable-Code bouclait sur « ### Instruction »	Mode complétion, mots d'arrêt, coupure des répétitions
3	Toutes les alertes placées à la ligne 1	Ligne réelle fournie par Bandit, doublons supprimés
4	Imports comptés comme vulnérabilités	Alertes B401–B413 traitées comme informatives
5	Observations identiques pour tous les modèles	Observations générées à partir des données
6	Trois CWE seulement	15 prompts couvrant 15 CWE distinctes
7	Accents corrompus dans les fichiers	Écriture systématique en UTF-8

Conclusion

L'implémentation tient sur un GPU gratuit grâce au chargement séquentiel des modèles et à la quantification 4 bits, et chaque résultat reste traçable jusqu'à son fichier source. Le chapitre suivant présente les résultats obtenus.

Chapitre 5

Résultats et discussion

Introduction

Ce chapitre répond aux quatre questions de recherche à partir de 5 modèles, 74 codes générés exploitables, 76 exemples de test étiquetés, 20 configurations de détecteurs et 105 tentatives de correction, puis discute les limites de l'étude.

5.1 Q1 — Sécurité du code généré

Table 5.1 – Vulnérabilités détectées par Bandit dans le code généré (alertes réelles MEDIUM+)

Modèle	Vulnérables	Taux	HIGH/MED./LOW	Latence (s)	VRAM (Go)
Qwen2.5-Coder 1.5B	4 / 15	26,7 %	2 / 3 / 2	3,9	3,1
Qwen2.5-Coder 3B	6 / 15	**40,0 %**	3 / 3 / 3	6,1	6,2
StarCoder2 3B	4 / 15	26,7 %	2 / 4 / 3	9,8	6,1
Stable-Code 3B	4 / 14	28,6 %	0 / 5 / 6	9,5	5,8
TinyLlama 1.1B	4 / 15	26,7 %	2 / 3 / 4	8,9	2,2
Total	**22 / 74**	**29,7 %**			

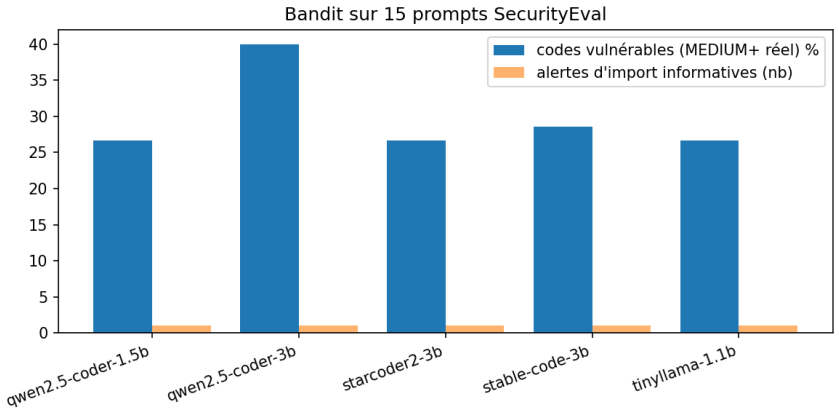


Figure 5.1 – Taux de codes vulnérables par modèle (Bandit, 15 prompts SecurityEval)

Tous les modèles produisent du code vulnérable. Les failles les plus fréquentes relèvent de « réflexes d'API » : désérialisation avec `pickle`, mode debug de Flask, module `random` pour des secrets, mots de passe écrits en dur. Les prompts CWE-020, CWE-022, CWE-259 et CWE-330 déclenchent des alertes Bandit chez les cinq modèles. Avec 15 codes par modèle, un code de différence représente toutefois près de 7 points : les écarts entre modèles doivent être interprétés avec prudence.

5.2 Q2 — Détection des vulnérabilités

Table 5.2 – Score F1 des détecteurs sur le test étiqueté (76 exemples)

Détecteur	**Base**	**Base + RAG**	**LoRA**	**LoRA + RAG**
Qwen2.5-Coder 1.5B	0,00	0,00	0,67	**0,77**
Qwen2.5-Coder 3B	0,04	0,10	**0,81**	0,69
StarCoder2 3B	0,00	0,00	**0,75**	0,67
Stable-Code 3B	0,10	0,00	**0,78**	0,49
TinyLlama 1.1B	0,00	0,00	**0,78**	0,05
Bandit (référence)			*0,64*	

Table 5.3 – Détail de la configuration LoRA et de Bandit

Détecteur	**Exact.**	**Préc.**	**Rappel**	**F1 [IC 95 %]**	**MCC**	**CWE juste**
Qwen2.5-Coder 1.5B	0,70	0,74	0,61	0,67 [0,52 – 0,79]	0,40	100 %
Qwen2.5-Coder 3B	0,80	0,77	0,87	0,81 [0,71 – 0,89]	0,61	88 %
StarCoder2 3B	0,75	0,74	0,76	0,75 [0,64 – 0,85]	0,50	79 %
Stable-Code 3B	0,75	0,69	0,89	0,78 [0,67 – 0,87]	0,52	85 %
TinyLlama 1.1B	0,78	0,77	0,79	0,78 [0,67 – 0,88]	0,55	76 %
Bandit	0,59	0,57	0,71	0,64 [0,51 – 0,74]	0,19	41 %

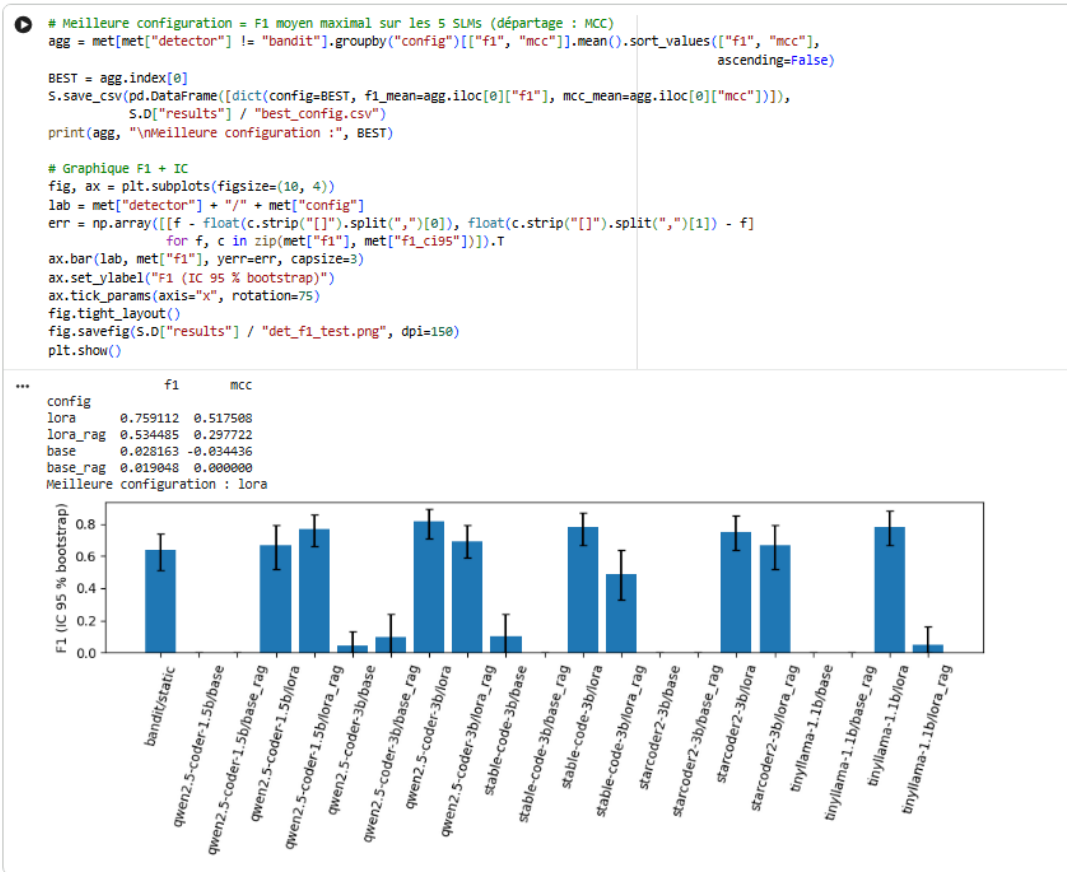


Figure 5.2 – Notebook 04 — F1 par détecteur et configuration, avec intervalles de confiance

— **Les modèles de base ne détectent pas** : ils répondent presque toujours « sûr », d'où un F1 proche de 0.

— **L'affinage LoRA est décisif** : le gain par rapport au modèle de base est significatif pour les cinq modèles (McNemar, $p \leq 0{,}02$).

— **LoRA dépasse Bandit** pour les cinq modèles, de façon significative pour quatre d'entre eux ($p < 0{,}04$; Qwen2.5-Coder 1.5B : $p = 0{,}19$).

— **Le RAG n'aide qu'un modèle** (Qwen2.5-Coder 1.5B) et dégrade les autres, significativement pour Qwen2.5-Coder 3B ($p = 0{,}001$) et TinyLlama ($p = 0{,}002$). Le contexte n'ayant pas été vu pendant l'entraînement, il décale la distribution d'entrée.

— Toutes les configurations produisent un JSON valide (0 % de réponses inutilisables).

5.3 Rôles échangés : générateurs et détecteurs

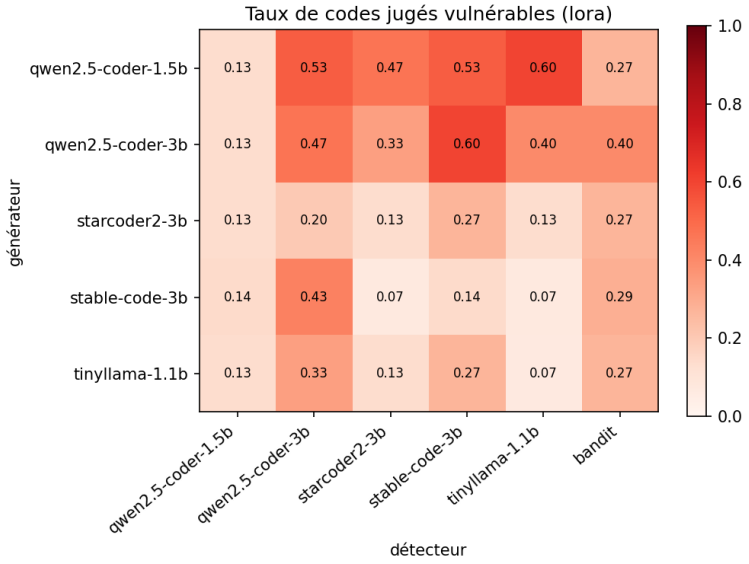


Figure 5.3 – Taux de codes jugés vulnérables, par générateur (lignes) et par détecteur (colonnes), configuration LoRA

L'accord entre les six détecteurs est modéré (kappa de Fleiss 0,36 ; kappa de Cohen moyen 0,35, figure 5.4) : ils ne signalent pas les mêmes codes. Trois détecteurs se montrent plus indulgents envers leur propre code : StarCoder2 (13 % contre 25 % pour le code des autres), Stable-Code (14 % contre 42 %) et TinyLlama (7 % contre 31 %) ; Qwen2.5-Coder 3B fait exception (47 % contre 37 %).

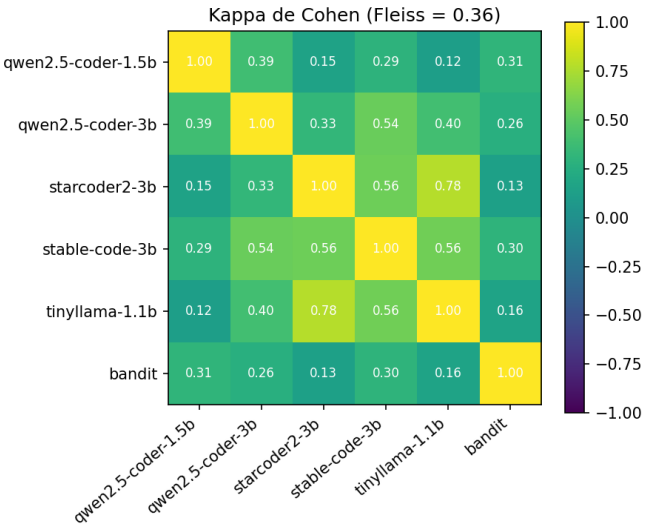


Figure 5.4 – Accord deux à deux entre détecteurs (kappa de Cohen)

5.4 Q3 — Correction automatique

Table 5.4 – Résultats de la correction par correcteur (21 tentatives chacun)

Correcteur	Succès brut	Succès strict	Validés (humain)	Taux réel
Qwen2.5-Coder 1.5B	14 %	0	0	0,0 %
Qwen2.5-Coder 3B	43 %	4	2	9,5 %
StarCoder2 3B	52 %	7	1	4,8 %
Stable-Code 3B	38 %	4	2	9,5 %
TinyLlama 1.1B	19 %	2	0	0,0 %
Total (105)	**33 % (35)**	**17**	**5**	**4,8 %**

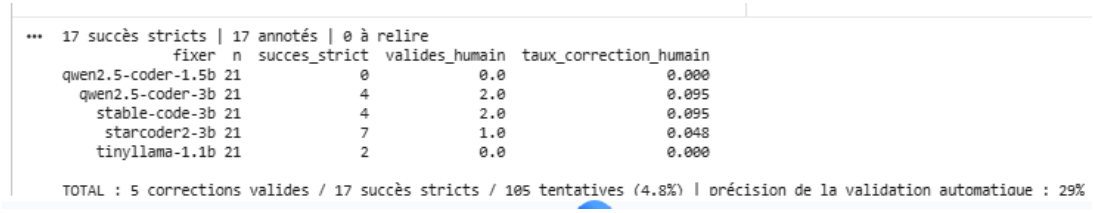


Figure 5.5 – Notebook 05 — revue humaine des 17 succès stricts

Les cinq corrections confirmées sont des remplacements locaux d'API : `debug=True` remplacé par `debug=False` (deux fois), `mktemp()` remplacé par `TemporaryFile` ou `NamedTemporaryFile` (deux fois), `random` remplacé par `secrets` (une fois). Les douze autres passaient tous les contrôles automatiques sans corriger la faille : protection XML inopérante (`remove_blank_text`), liste noire contournable pour `pickle`, en-tête `Content-Type` toujours contrôlé par l'utilisateur, mot de passe supprimé au lieu d'être lu dans l'environnement, nom de fichier temporaire fixe, simple changement cosmétique d'exception. La précision de la validation automatique n'est donc que d'environ **29 %** (5 sur 17).

La correction croisée réussit mieux que l'auto-correction (17,9 % de succès stricts contre 9,5 %), ce qui rejoint l'indulgence observée en détection. Par ailleurs, 24 % des tentatives renvoient un code inchangé.

5.5 Analyse des erreurs de détection

Sur le test étiqueté, les faux positifs concernent du code sûr contenant des API sensibles : `subprocess` apparaît dans 65 % des faux positifs contre 10 % des prédictions correctes, `pickle` dans 63 % contre 19 %. Les détecteurs réagissent au vocabulaire plutôt qu'à l'usage. Les faux négatifs se concentrent sur des failles qui dépendent du flux des données : CWE-209 (10 cas), CWE-89 (8) et CWE-120 (7).

La localisation de la ligne vulnérable (référence : lignes Bandit, tolérance ±1, 35 codes) reste limitée : Stable-Code 46 %, Qwen2.5-Coder 1.5B et 3B 31 %, StarCoder2 et TinyLlama 0 %, ces

deux derniers ne renvoyant pas de numéro de ligne exploitable.

5.6 Q4 — Taille, performance fonctionnelle et sécurité

Table 5.5 – Corrélations de rang (Spearman)

Lien testé	n	ρ	p
Taille et taux vulnérable (Bandit)	5	0,67	0,22
Taille et taux vulnérable (pseudo-vérité)	5	0,15	0,81
pass@1 et taux vulnérable (pseudo-vérité)	4	0,60	0,40
Taille et F1 de détection	5	0,50	0,39

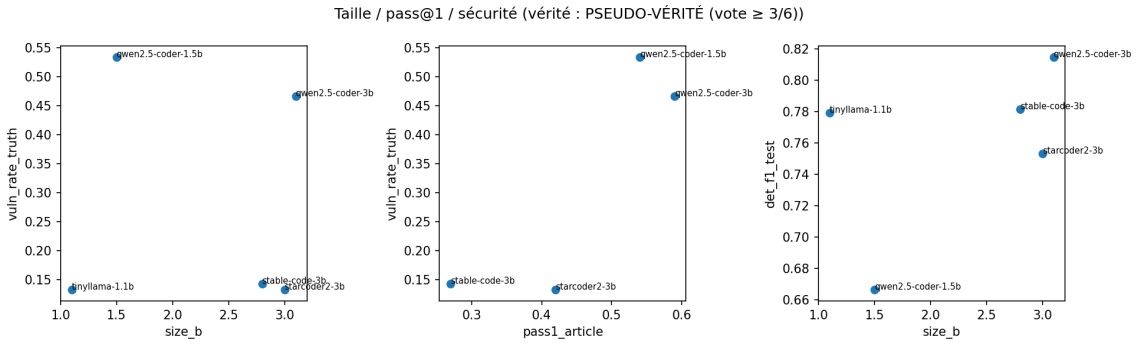
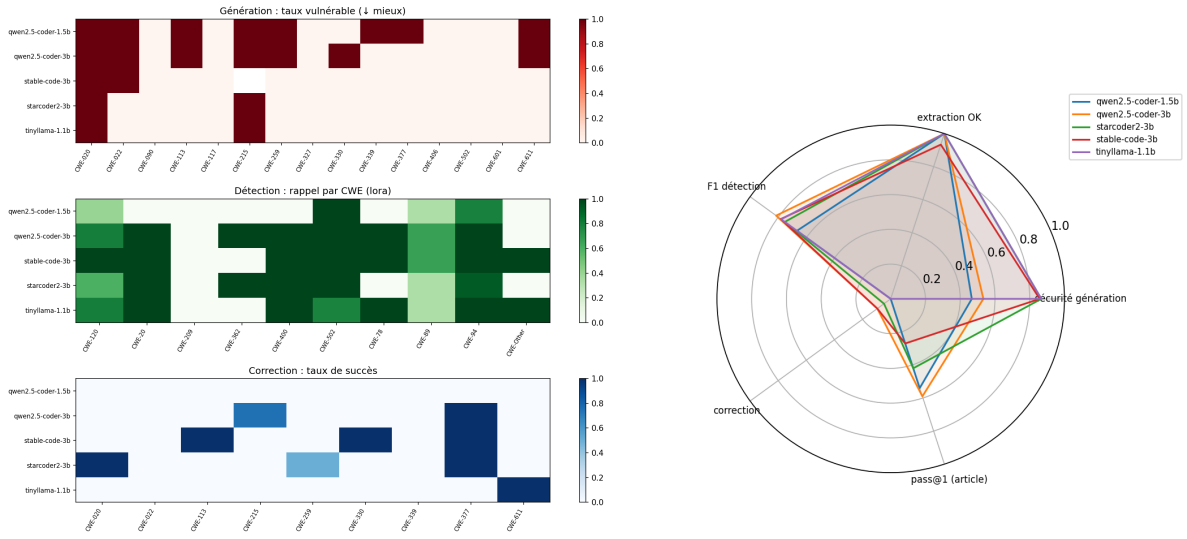


Figure 5.6 – Taille, pass@1 et sécurité (pseudo-vérité : vote $\geq 3/6$)

Aucune corrélation n'est significative avec quatre ou cinq modèles. Aucune ne va cependant dans le sens « plus grand ou plus performant = plus sûr » : Qwen2.5-Coder 3B, meilleur pass@1 (0,59), présente le taux de vulnérabilité Bandit le plus élevé (40 %), et TinyLlama (1,1 B), une fois affiné, détecte aussi bien que les modèles de 3 milliards de paramètres.

5.7 Profils de compétences



(a) Par modèle et par CWE **(b)** Synthèse par modèle

Figure 5.7 – Profils de compétences : génération, détection et correction

5.8 Discussion

Génération

Près d'un code sur trois est vulnérable, principalement par reproduction d'API courantes mais dangereuses. La qualité fonctionnelle mesurée par pass@1 ne protège pas de ces failles.

Détection

Un affinage QLoRA léger suffit à transformer un SLM en détecteur plus performant que Bandit, capable en outre d'identifier la bonne catégorie de faille : un détecteur local, peu coûteux et explicable.

RAG

Le RAG n'est pas un gain automatique : sans entraînement adapté au contexte, il perturbe le détecteur.

Correction

La correction automatique reste peu fiable, et les contrôles automatiques surestiment fortement sa réussite : une relecture humaine est indispensable.

5.9 Limites

— **Taille d'échantillon** : 15 prompts et une réponse par prompt ; les écarts entre générateurs ne sont pas statistiquement robustes.

— **Pseudo-vérité** : sur les codes générés, la vérité est un vote des détecteurs, donc biaisée en leur faveur ; le fichier d'annotation humaine est prêt mais n'a pas été rempli.

— **Jeu d'entraînement** : CyberNative est petit (594 exemples) et synthétique, et ses CWE sont déduites par mots-clés ; les scores sur le test surestiment probablement la performance sur du code réel.

— **RAG absent de l'entraînement** : il explique la dégradation des configurations LoRA + RAG.

— **Bandit** repère des motifs d'appels mais ignore les failles logiques et le flux des données.

— **Pas de tests fonctionnels** : la préservation du comportement des corrections n'est vérifiée que par la relecture humaine.

— **pass@1** provient d'un autre protocole, et TinyLlama n'y figure pas.

Conclusion

Les résultats montrent un contraste net entre les trois rôles : les SLMs génèrent régulièrement du code vulnérable, deviennent de bons détecteurs après un affinage léger, mais restent de mauvais correcteurs.

Conclusion générale et perspectives

Ce projet a évalué cinq SLMs open source dans trois rôles complémentaires, sur un GPU gratuit et avec un pipeline entièrement reproductible. Il montre que :

1. **les SLMs génèrent du code vulnérable** dans **29,7 %** des cas, et un meilleur score fonctionnel ne garantit pas un code plus sûr ;
2. **un SLM affiné par QLoRA**, en quelques minutes, détecte les vulnérabilités **mieux que Bandit** (F1 jusqu'à **0,81** contre 0,64) et identifie mieux leur catégorie ;
3. **la correction automatique reste peu fiable** : **4,8 %** seulement des tentatives aboutissent à une correction réellement valide, et la validation automatique ne détecte pas la plupart des fausses corrections.

En résumé, **un SLM peut surveiller le code, mais pas encore le réparer seul**. Un détecteur local, léger et explicable est à portée de main, à condition de conserver un humain dans la boucle pour la correction.

Perspectives.

— Annoter manuellement les 74 codes générés pour disposer d'une vérité indépendante.

— Générer plusieurs réponses par prompt pour obtenir des taux statistiquement solides.

— Entraîner les détecteurs avec le contexte RAG.

— Utiliser un jeu d'entraînement plus réaliste, issu de vulnérabilités réelles.

— Ajouter des tests unitaires pour valider le comportement des corrections.

— Étendre l'étude à d'autres langages de programmation.

Annexe A

Glossaire

Adaptateur LoRA
Petit ensemble de poids entraînés, ajouté à un modèle figé pour le spécialiser.

Alerte informative
Alerte Bandit déclenchée par un simple import (B401 à B413), non comptée comme vulnérabilité.

Embedding
Représentation vectorielle d'un texte permettant de mesurer sa proximité avec d'autres textes.

Pseudo-vérité
Étiquette obtenue par vote majoritaire de plusieurs détecteurs (ici au moins 3 sur 6), en l'absence d'annotation humaine.

Quantification
Représentation des poids d'un modèle avec moins de bits (ici 4) pour réduire la mémoire nécessaire.

Succès strict
Correction qui passe tous les contrôles automatiques et tous les garde-fous.

Token
Unité de texte (mot, morceau de mot ou symbole) manipulée par un modèle de langage.

Zero-shot
Utilisation d'un modèle sans exemple ni entraînement spécifique à la tâche.

Annexe B

Acronymes

AST	*Abstract Syntax Tree*, arbre syntaxique abstrait
CWE	*Common Weakness Enumeration*
FAISS	*Facebook AI Similarity Search*
F1	Moyenne harmonique de la précision et du rappel
GPU	*Graphics Processing Unit*
IC	Intervalle de confiance
INPT	Institut National des Postes et Télécommunications
JSON	*JavaScript Object Notation*
LoRA	*Low-Rank Adaptation*
MCC	*Matthews Correlation Coefficient*
NF4	*NormalFloat 4 bits*
QLoRA	*Quantized LoRA*
RAG	*Retrieval-Augmented Generation*
SLM	*Small Language Model*
VRAM	*Video Random Access Memory*, mémoire du GPU

Annexe C

Reproduction des expériences

C.1 Dépôt du projet

Le code, les notebooks, les résultats, ce rapport et la présentation sont disponibles sur GitHub : https://github.com/manelell83/slm-security.

Table C.1 – Organisation du dépôt

`slmsec.py`	bibliothèque commune
`notebooks/`	une phase par notebook Google Colab (00 à 06)
`notebooks_src/`	sources des notebooks, régénérés par `tools/build_notebooks.py`
`results/`	tableaux CSV et figures produits par les notebooks
`rapport/`	ce rapport (LaTeX)
`presentation/`	présentation de soutenance (HTML) et mode présentateur

C.2 Ordre d'exécution

1. Copier `slmsec.py` dans `MyDrive/slm-security/`.
2. Exécuter dans l'ordre les notebooks 00 (configuration), 01 (génération), 02 (Bandit), 03 (détecteurs), 04 (benchmark), 05 (correction) et 06 (extras), sur un environnement Colab avec GPU T4.
3. Chaque étape enregistre ses résultats sur Drive et saute ce qui existe déjà : un notebook interrompu peut être relancé.

Bibliographie

[1] M. M. Hasan, M. Waseem, K.-K. Kemell, J. Rasku, J. Ala-Rantala et P. Abrahamsson. *Assessing small language models for code generation : An empirical study with benchmarks*. The Journal of Systems & Software, 2026.

[2] M. L. Siddiq et J. C. S. Santos. *SecurityEval dataset : mining vulnerability examples to evaluate machine learning-based code generation techniques*. MSR4P&S Workshop, 2022.

[3] CyberNative. *Code_Vulnerability_Security_DPO* (jeu de données). Hugging Face, https://huggingface.co/datasets/CyberNative/Code_Vulnerability_Security_DPO.

[4] MITRE. *Common Weakness Enumeration (CWE)*. https://cwe.mitre.org.

[5] PyCQA. *Bandit — security linter for Python*. https://github.com/PyCQA/bandit.

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang et W. Chen. *LoRA : Low-Rank Adaptation of Large Language Models*. ICLR, 2022.

[7] T. Dettmers, A. Pagnoni, A. Holtzman et L. Zettlemoyer. *QLoRA : Efficient Finetuning of Quantized LLMs*. NeurIPS, 2023.

[8] P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.

[9] S. Xiao, Z. Liu, P. Zhang et N. Muennighoff. *C-Pack : Packaged Resources To Advance General Chinese Embedding*. arXiv :2309.07597, 2023.

[10] J. Johnson, M. Douze et H. Jégou. *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data, 2019.

[11] T. Wolf et al. *Transformers : State-of-the-Art Natural Language Processing*. EMNLP : System Demonstrations, 2020.